

Table des matières

Introduction

Système d'exploitation

Ligne de commande

Processus et fils d'exécution

Utilisateurs et groupes

Fichiers

Gestion de la mémoire

Concurrence et parallélisme

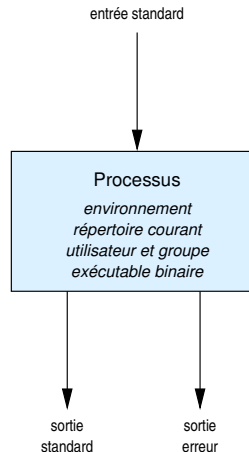
Compilation

Environnement logiciel et virtualisation

Processus

Définition

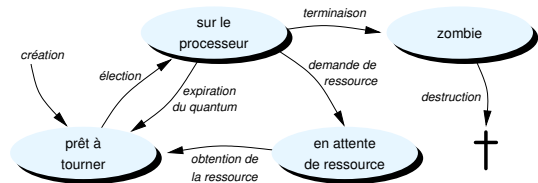
- instance d'un programme à l'instant t
- identifié par un numéro unique
 - *Process Identifier* (PID)
- composé d'un ou plusieurs fils d'exécution
- connaît plusieurs états d'existence
- peut avoir un parent
- peut engendrer des processus fils
- dispose d'un tas et d'une pile à travers d'une vision unique de la mémoire du système (mémoire virtuelle)
- dispose d'un espace mémoire et des ressources système qui lui sont propres



Processus

Définition

- instance d'un programme à l'instant t
- identifié par un numéro unique
 - *Process Identifier* (PID)
- composé d'un ou plusieurs fils d'exécution
- connaît plusieurs états d'existence
- peut avoir un parent
- peut engendrer des processus fils
- dispose d'un tas et d'une pile à travers d'une vision unique de la mémoire du système (mémoire virtuelle)
- dispose d'un espace mémoire et des ressources système qui lui sont propres



Processus

Définition

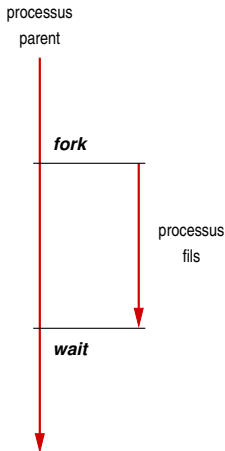
- instance d'un programme à l'instant t
- identifié par un numéro unique
 - *Process Identifier* (PID)
- composé d'un ou plusieurs fils d'exécution
- connaît plusieurs états d'existence
- peut avoir un parent
- peut engendrer des processus fils
- dispose d'un tas et d'une pile à travers d'une vision unique de la mémoire du système (mémoire virtuelle)
- dispose d'un espace mémoire et des ressources système qui lui sont propres

```
$ pstree
systemd---ModemManager
          |--NetworkManager
          |--accounts-daemon
          |--agetty
          |--apache2
          |--atril
          |--avahi-daemon
          |--blueman-tray
          |--bluetoothd
          |--boltd
          |--colord
          |--containerd
          |--crash_handler
          |--cron
          |--cups-browsed
          |--cupsd
          |--dbus-daemon
          |--dockerd
          |--emacs
          ...
```

Processus

Création

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
int main(int argc, char ** argv) {
    pid_t child = fork();
    if(child == 0) {
        printf("CHILD\n");
        fflush(stdout);
        sleep(3);
        printf("CHILD: done waiting\n");
        fflush(stdout);
    } else {
        printf("PARENT\n");
        fflush(stdout);
        wait(NULL);
        printf("PARENT: child done\n");
        fflush(stdout);
    }
    return 0;
}
```



Processus

Création

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
int main(int argc, char ** argv) {
    pid_t child = fork();
    if(child == 0) {
        printf("CHILD\n");
        fflush(stdout);
        sleep(3);
        printf("CHILD: done waiting\n");
        fflush(stdout);
    } else {
        printf("PARENT\n");
        fflush(stdout);
        wait(NULL);
        printf("PARENT: child done\n");
        fflush(stdout);
    }
    return 0;
}
```

PARENT

CHILD

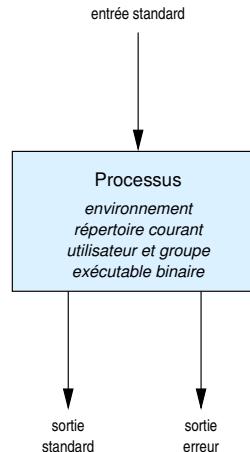
CHILD: done waiting

PARENT: child done

Fils d'exécution

Définition

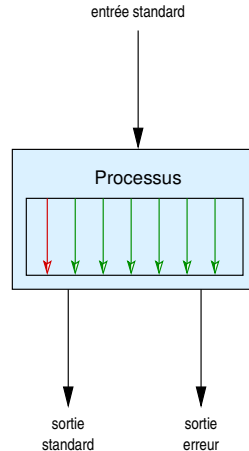
- fait partie d'un processus
- identifié par un numéro unique – *Thread Identifier* (TID)
- dispose de sa propre pile, mais partage la vision de la mémoire du processus parent
- peut s'exécuter de façon concurrente avec les autres fils d'exécution du processus parent sur une architecture multi-cœur
- hérite des fichiers ouverts de son parent



Fils d'exécution

Définition

- fait partie d'un processus
- identifié par un numéro unique – *Thread Identifier* (TID)
- dispose de sa propre pile, mais partage la vision de la mémoire du processus parent
- peut s'exécuter de façon concurrente avec les autres fils d'exécution du processus parent sur une architecture multi-cœur
- hérite des fichiers ouverts de son parent



Fils d'exécution

Définition

PID	USER	PRI	NI	UIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
2895	marek	20	0	453M	52264	40720	S	0.0	0.2	0:06.58	/usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-
2904	marek	20	0	453M	52264	40720	S	0.0	0.2	0:00.00	usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-
2905	marek	20	0	453M	52264	40720	S	0.0	0.2	0:00.00	usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-
2907	marek	20	0	453M	52264	40720	S	0.0	0.2	0:00.57	usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-
2938	marek	20	0	453M	52264	40720	S	0.0	0.2	0:00.02	usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-
2911	marek	20	0	435M	42060	33384	S	0.0	0.1	0:00.04	xfce4-panel
75094	marek	20	0	3783M	567M	257M	S	0.0	1.8	3:49.49	/opt/firefox/firefox
75097	marek	20	0	3783M	567M	257M	S	0.0	1.8	0:00.00	/opt/firefox/firefox
75101	marek	20	0	3783M	567M	257M	S	0.0	1.8	0:00.00	/opt/firefox/firefox
75102	marek	20	0	3783M	567M	257M	S	0.0	1.8	0:00.00	/opt/firefox/firefox
75104	marek	20	0	3783M	567M	257M	S	0.0	1.8	0:00.08	/opt/firefox/firefox
75105	marek	20	0	3783M	567M	257M	S	0.0	1.8	0:40.85	/opt/firefox/firefox
75107	marek	20	0	3783M	567M	257M	S	0.0	1.8	1:11.25	/opt/firefox/firefox
75108	marek	20	0	3783M	567M	257M	S	0.0	1.8	0:06.19	/opt/firefox/firefox
75109	marek	20	0	3783M	567M	257M	S	0.0	1.8	0:00.20	/opt/firefox/firefox
75110	marek	20	0	3783M	567M	257M	S	0.0	1.8	0:00.57	/opt/firefox/firefox
75111	marek	20	0	3783M	567M	257M	S	0.0	1.8	1:19.74	/opt/firefox/firefox
75113	marek	20	0	3783M	567M	257M	S	0.0	1.8	0:00.00	/opt/firefox/firefox

Fils d'exécution

Création

```
#include <unistd.h>
#include <stdio.h>
#include <pthread.h>
void * work(void * tid) {
    int me = *((int *) tid);
    printf("Thread %d starts\n", me);
    sleep(3);
    fflush(stdout);
}
int main(int argc, char ** argv) {
    pthread_t tid;
    int child = 1;
    pthread_create(&tid, NULL, work, (void *) &child);
    printf("Thread 0 starts\n");
    fflush(stdout);
    pthread_join(tid, NULL);
    printf("Thread 0 finishes\n");
    return 0;
}
```

fil d'exécution
principal

create

fil d'exécution
fils

join

Fils d'exécution

Création

```
#include <unistd.h>
#include <stdio.h>
#include <pthread.h>
void * work(void * tid) {
    int me = *((int *) tid);
    printf("Thread %d starts\n", me);
    sleep(3);
    fflush(stdout);
}
int main(int argc, char ** argv) {
    pthread_t tid;
    int child = 1;
    pthread_create(&tid, NULL, work, (void *) &child);
    printf("Thread 0 starts\n");
    fflush(stdout);
    pthread_join(tid, NULL);
    printf("Thread 0 finishes\n");
    return 0;
}
```

Thread 0 starts

Thread 1 starts

Thread 1 finishes

Thread 0 finishes

Processus fils versus fil d'exécution

Différence illustrée

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
int main(int argc, char ** argv) {
    int val = 2;
    pid_t child = fork();
    if(child == 0) { val += 1; }
    else {
        val += 2; wait(NULL);
    }
    printf("PID %d PPID %d VAL is %d\n",
        ↪ getpid(), getppid(), val);
    return 0;
}
```

Processus fils versus fil d'exécution

Différence illustrée

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
int main(int argc, char ** argv) {
    int val = 2;
    pid_t child = fork();
    if(child == 0) { val += 1; }
    else {
        val += 2; wait(NULL);
    }
    printf("PID %d PPID %d VAL is %d\n",
        ↪ getpid(), getppid(), val);
    return 0;
}
```

PID 139544 PPID 139538 VAL is 3

PID 139538 PPID 138810 VAL is 4

Processus fils versus fil d'exécution

Différence illustrée

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
int main(int argc, char ** argv) {
    int val = 2;
    pid_t child = fork();
    if(child == 0) { val += 1; }
    else {
        val += 2; wait(NULL);
    }
    printf("PID %d PPID %d VAL is %d\n",
        ↪ getpid(), getppid(), val);
    return 0;
}
```

PID 139544 PPID 139538 VAL is 3

PID 139538 PPID 138810 VAL is 4

```
#include <unistd.h>
#include <stdio.h>
#include <pthread.h>
void * work(void * arg) {
    int * val = (int *) arg; (*val) += 1;
}
int main(int argc, char ** argv) {
    int val = 2; pthread_t tid;
    pthread_create(&tid, NULL, work, (void *)
        ↪ &val);
    val += 2;
    pthread_join(tid, NULL);
    printf("PID %d PPID %d VAL is %d\n",
        ↪ getpid(), getppid(), val);
    return 0;
}
```

Processus fils versus fil d'exécution

Différence illustrée

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
int main(int argc, char ** argv) {
    int val = 2;
    pid_t child = fork();
    if(child == 0) { val += 1; }
    else {
        val += 2; wait(NULL);
    }
    printf("PID %d PPID %d VAL is %d\n",
        ↪ getpid(), getppid(), val);
    return 0;
}
```

PID 139544 PPID 139538 VAL is 3

PID 139538 PPID 138810 VAL is 4

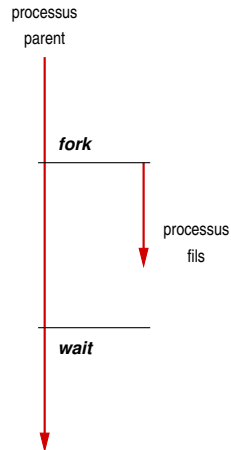
```
#include <unistd.h>
#include <stdio.h>
#include <pthread.h>
void * work(void * arg) {
    int * val = (int *) arg; (*val) += 1;
}
int main(int argc, char ** argv) {
    int val = 2; pthread_t tid;
    pthread_create(&tid, NULL, work, (void *)
        ↪ &val);
    val += 2;
    pthread_join(tid, NULL);
    printf("PID %d PPID %d VAL is %d\n",
        ↪ getpid(), getppid(), val);
    return 0;
}
```

PID 139662 PPID 138810 VAL is 5

Signaux

Définition

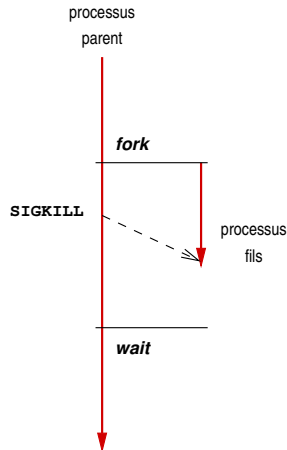
- **interruptions** logicielles asynchrones envoyées à un programme
- déclenchent l'arrêt du programme ou un traitement spécifique
- moyen de communication entre processus, entre processus et ses fils d'exécution ou entre noyau et processus



Signaux

Définition

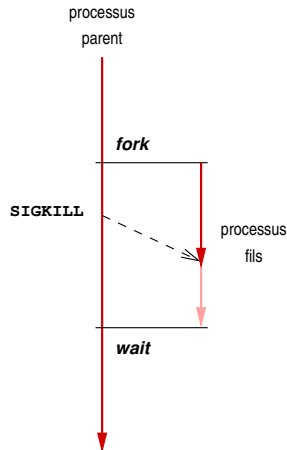
- **interruptions** logicielles asynchrones envoyées à un programme
- déclenchent l'arrêt du programme ou un traitement spécifique
- moyen de communication entre processus, entre processus et ses fils d'exécution ou entre noyau et processus



Signaux

Définition

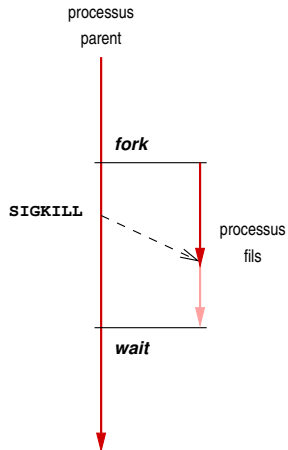
- **interruptions** logicielles asynchrones envoyées à un programme
- déclenchent l'arrêt du programme ou un traitement spécifique
- moyen de communication entre processus, entre processus et ses fils d'exécution ou entre noyau et processus



Signaux

Définition

Code	Nom du signal	Signification
1	SIGHUP	arrêter (fermeture du terminal)
2	SIGINT	interrompre (Ctrl+C)
8	SIGFPE	erreur arithmétique (div. par zéro, ...)
9	SIGKILL	termination forcée
10	SIGUSR1	signal utilisateur
11	SIGSEGV	erreur de segmentation
12	SIGUSR2	signal utilisateur
13	SIGPIPE	écriture sur un tube sans lecteur
14	SIGALRM	minuterie expirée
15	SIGTERM	demande de terminaison
28	SIGWINCH	redimensionnement de la fenêtre du terminal



Signaux

Manipulation

- représentés par des valeurs entières (1 à 31, resp. SIGHUP à SIGSYS)
- manipulés à travers des ensembles

signal.h

- vider un ensemble de signaux

```
int sigemptyset(sigset_t * set);
```

- remplir un ensemble de signaux

```
int sigfillset(sigset_t * set);
```

- ajouter un signal dans un ensemble de signaux

```
int sigaddset(sigset_t * set, int signum);
```

- retirer un signal d'un ensemble de signaux

```
int sigdelset(sigset_t * set, int signum);
```

- vérifier l'appartenance d'un signal à un ensemble de signaux

```
int sigismember(const sigset_t *set, int signum);
```

Signaux

Manipulation

Blocage

```
int sigprocmask(int how, const sigset_t * set, sigset_t * oldset);
```

- possibilité de bloquer temporairement des signaux (sauf SIGKILL)
 - par exemple, avant d'entrer dans une section critique

```
sigset_t blk;  
sigset_t sigsv;  
sigemptyset(&blk);  
sigaddset(&blk, SIGINT);  
sigaddset(&blk, SIGPIPE);  
sigprocmask(SIG_BLOCK, &blk, &sigsv);  
/* section critique */  
sigprocmask(SIG_SETMASK, &sigsv, NULL);
```

Signaux

Manipulation

Récupération

```
int sigpending(const sigset_t * set);
```

- possibilité de récupérer l'ensemble des signaux précédemment bloqués

```
sigset_t pending;  
sigpending(&pending);  
if(sigismember(&pending, SIGINT)) {  
    puts("SIGINT is pending");  
}
```

Signaux

Manipulation

Déclenchement

- possibilité de déclencher les signaux précédemment bloqués après leur récupération
- rétablissement du blocage à la fin du traitement

```
sigset_t pending;
sigset_t notpipe;
sigfillset(&notpipe);
sigdelset(&notpipe, SIGPIPE);
sigpending(&pending);
if(sigismember(&pending, SIGPIPE)) {
    sigsuspend(&notpipe);
}
// rétablissement du masque des signaux
```

Signaux

Manipulation

Lever un signal

```
int kill(pid_t pid, int sig);
```

- possibilité d'envoyer un signal à n'importe quel processus

Minuterie

```
unsigned int alarm(unsigned int seconds);
```

- lever le signal SIGALRM au bout d'un certain nombre de secondes
- annulation d'une précédente minuterie avec alarm(0)

Signaux

Manipulation

- déclenchement d'une réponse spécifique à la réception par un processus ou un fil d'exécution

Code	Nom du signal	Signification	Action par défaut
1	SIGHUP	arrêter (fermeture du terminal)	terminer
2	SIGINT	interrompre (Ctrl+C)	terminer
8	SIGFPE	erreur arithmétique (div. par zéro, ...)	terminer ¹
9	SIGKILL	terminaison forcée	terminer
10	SIGUSR1	signal utilisateur	terminer
11	SIGSEGV	erreur de segmentation	terminer ¹
12	SIGUSR2	signal utilisateur	terminer
13	SIGPIPE	écriture sur un tube sans lecteur	terminer
14	SIGALRM	minuterie expirée	terminer
15	SIGTERM	demande de terminaison	terminer
28	SIGWINCH	redimensionnement de la fenêtre du terminal	ignorer

1. terminer et sauvegarder une empreinte de la mémoire—*core dump*

Signaux

Manipulation

Changement d'action associée

```
int sigaction(int signum, const struct sigaction * action, struct sigaction * oldaction);
struct sigaction {
    // pointeur vers la fonction à appeler à la réception du signal
    void (* sa_handler) (int signal);
    // alternative à `sa_handler' (si `sa_flags' contient `SA_SIGINFO')
    void (* sa_sigaction) (int signal, siginfo_t *, void *);
    sigset_t sa_mask; // signaux à bloquer lors du traitement du signal
    int sa_flags; // drapeaux pour modifier le comportement du signal
};
```

- redéfinir l'action associée à un signal (sauf pour SIGKILL et SIGSTOP)

Attention ! Tout n'est pas permis !
man signal-safety

Signaux

Manipulation

Exemple : redéfinir l'action du signal SIGINT

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
void handler(int sig) {
    printf("SIGINT intercepté\n");
    exit(EXIT_SUCCESS);
}
int main (int argc, char ** argv) {
    struct sigaction act = { .sa_handler = &handler, .sa_flags = 0 };
    sigemptyset(&act.sa_mask);
    if(sigaction(SIGINT, &act, NULL) < 0) {
        perror("sigaction");
        return 1;
    }
    while(1) sleep(10); // uniquement pour maintenir le processus en vie
    return 0;
}
```

Signaux

Manipulation

Exemple : afficher l'heure actuelle toutes les secondes

```
#include <stdio.h>
#include <signal.h>
#include <time.h>
#include <unistd.h>
#include <stdlib.h>

void report_time(int signal) {
    time_t curtime; time(&curtime);
    printf("current time is %s", ctime(&curtime));
    alarm(1); // réenclencher la minuterie
}

int main(void){
    struct sigaction act = { .sa_handler = &report_time, .sa_flags = 0 };
    sigemptyset(&act.sa_mask);
    if(sigaction(SIGALRM, &act, NULL) < 0) { perror("sigaction"); return 1; }
    alarm(1);
    while(1) sleep(10); // uniquement pour maintenir le processus en vie
    return 0;
}
```

Démarrage d'un programme

- lancement d'un programme en ligne de commande

```
mkdir -p /tmp/dir
```

- fonction principale main du programme lancé

```
int main(int argc, char ** argv) {  
    ...  
    return 0;  
}
```

Variable	Contenu
argc	3
argv[0]	« mkdir »
argv[1]	« -p »
argv[2]	« /tmp/dir »
argv[argc]	NULL

Démarrage d'un programme

- lancement d'un programme en ligne de commande

```
mkdir -p /tmp/dir
```

- fonction principale main du programme lancé

```
int main(int argc, char ** argv, char **
↪ envp) {
    ...
    return 0;
}
```

Variable	Contenu
argc	3
argv[0]	« mkdir »
argv[1]	« -p »
argv[2]	« /tmp/dir »
argv[argc]	NULL
envp[0]	« PATH=/usr/bin :... »
envp[1]	« PWD=/home/... »
...	...
envp[taille]	NULL

Exécution d'un programme

- exécution d'un programme depuis un autre programme

```
int execvp(const char * file, char * const argv[]);
```

- file – chemin de l'exécutable à lancer
- argv – liste des arguments à passer à l'exécutable

Variantes

```
int execl(const char *pathname, const char *arg, ... /*, NULL */);  
int execlp(const char *file, const char *arg, ... /*, NULL */);  
int execlxe(const char *pathname, const char *arg, ... /*, NULL, char *const envp[] */);  
int execv(const char *pathname, char *const argv[]);  
int execvp(const char *file, char *const argv[]);  
int execvpe(const char *file, char *const argv[], char *const envp[]);
```

Exécution d'un programme

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
int main(int argc, char ** argv) {
    pid_t child = fork();
    if(child == 0) {
        char * const prog[] = { "ls", ".", NULL };
        if(execvp(prog[0], prog) != 0) {
            perror("Reading current directory");
        }
    } else {
        wait(NULL);
    }
    return 0;
}
```

```
beamerthemeuvsq
figures
lecture.bbl
lecture.org
lecture.org~
lecture.pdf
lecture.tex
_minted-lecture
pictures
public
publish.el
publish.el~
```

Gestion en ligne de commande

- liste des processus de l'utilisateur courant dans la session courante

ps

PID	TTY	TIME	CMD
97442	pts/0	00 :00 :00	bash
102542	pts/0	00 :00 :00	ps

- liste des processus de tous les utilisateurs dans la session courante

ps -u

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
marek	97442	0.0	0.0	9008	5376	pts/0	Ss+	15 :24	0 :00	bash

- liste des processus de tous les utilisateurs dans toutes les sessions

ps -au

- liste de tous les processus, y compris ceux qui ne sont pas attachés à un terminal

ps -aux

Gestion en ligne de commande

- liste des signaux et de leurs codes

```
kill -l
```

- envoi d'un signal à un processus

```
kill <PID> # SIGINT, par défaut
```

```
kill -s <signal> <PID>
```

```
pkill <nom>
```

```
pkill -s <signal> <nom>
```

- lancement d'un processus en arrière plan

```
<exécutable> &
```

- attente des processus en arrière plan

```
wait
```

- liste des processus en arrière plan

```
jobs
```